



Sparser MoE Explorations

Router, communication, and systems
co-design at 2,304 experts

Harry Hanyu

Internship final presentation | July 2026



Presentation outline

Introduction

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization
- 5 HybridEP Optimization
- 6 MCore Integration
- 7 End-to-End Results

Sparser MoE Trends

Sparser routing is one possible scaling direction, not a universal design

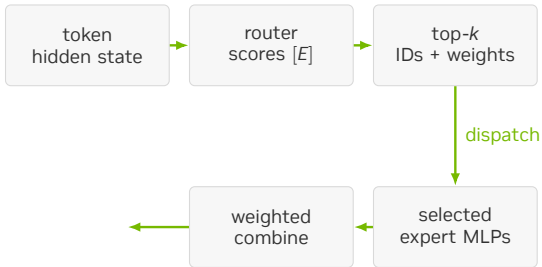
Open model	Scale	Routed experts	Selected/token
Mixtral 8×7B	Early open sparse MoE	8	2
DeepSeek-V3	671B total / 37B active	256	8
Nemotron 3 Ultra	550B total / 55B active	512	22
DeepSeek-V4-Pro	1.6T total / 49B active	384	6
Kimi K2	1T total / 32B active	384	8
Kimi K3	2.8T total / 104B active	896	16

Trend

LatentMoE reduces the routed representation before expert computation. It provides one path to increase routed-expert count or top-*k* while controlling the sparse-compute budget.

MoE Execution

Only selected experts execute, but every token still routes



```
scores = router(x)           # E scores
ids, weights = topk(scores, K)

y = 0
for j in range(K):
    y += weights[j] * expert[ids[j]](x)
```

Non-MLP work

Sparse compute reduces MLP FLOPs but does not eliminate owner selection, token exchange, weight preservation, or reverse-path work in backward.

LatentMoE Cost Model

Router and communication costs decrease more slowly than expert computation

Conventional experts

$$\text{expert work} \propto T \times K \times H \times H_{\text{ffn}}$$

Large expert GEMMs amortize routing, launch, and metadata overhead.

Latent experts

$$H_{\text{ffn}} \downarrow \Rightarrow \text{expert time} \downarrow$$

Router remains $T \times E$; ownership and token exchange remain $T \times K$.

Sparse-path budget

The relevant optimization unit is the complete sparse path: selection, compact metadata, dispatch preprocessing, communication, expert compute, and combine.

Target Workload



Presentation outline

Target Workload

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization
- 5 HybridEP Optimization
- 6 MCore Integration
- 7 End-to-End Results

Target Configuration

The 2,304-expert test exposes costs that are not visible in the public 512-expert configuration

Dimension	Public 80B	2,304E test
Routed experts	512	2,304
Selected experts	10	36
MoE intermediate	512	512
Expert parallelism	–	72
Local experts/rank	–	32
Layers	48	24

4.5×

router-score
count

512 → 2,304
experts

Runtime envelope

3.6×

selected-route
count

10 → 36

TP1 / PP1 / EP72, MBS 3, GBS 864,
MXFP8, 1F1B, full-iteration CUDA
graph.

Initial Analysis



Presentation outline

Initial Analysis

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis**
- 4 Fused-Router Optimization
- 5 HybridEP Optimization
- 6 MCore Integration
- 7 End-to-End Results

Initial Profile

A historical snapshot exposed the critical path

Region	Time
Attention	3.1 ms
HybridEP dispatch	9.7 ms
Expert MLP	1.1 ms
HybridEP combine	2.5 ms

Principal bottlenecks

1. CPU launch overhead from many small kernels.
2. Fused-router degradation at large E and K .
3. Communication and preprocessing overhead.

Eliminate launch overhead before analyzing the replay-stable GPU critical path.

Launch-Overhead Mitigation

Make one iteration replayable, then optimize the remaining GPU bottlenecks

```
# capture once
for microbatch in iteration:
    route()
    dispatch()
    grouped_mlp()
    combine()

# replay without Python launch gaps
cuda_graph.replay()
```

- Increase microbatch size where memory permits.
- Capture the full iteration, not isolated kernels.
- Use fused grouped MLP and related MoE kernels.
- Fuse metadata and dispatcher preprocessing kernels.

Router Optimization



Presentation outline

Fused-Router Optimization

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization**
- 5 HybridEP Optimization
- 6 MCore Integration
- 7 End-to-End Results

Top- k Selection

Radix selection replaces K -dependent scans with a fixed-pass algorithm

Baseline selection

```
for rank in range(K):  
    ids[rank] = argmax(scores) # scan E  
    remove(ids[rank])
```

Repeated work grows with $E \times K$

45.36 ms

pre-radix forward

T=65,536; E/K=2,304/36

Radix selection

```
prefix = 0  
for nibble in reversed(range(8)):  
    hist = histogram(scores, prefix)  
    prefix = bucket_containing_rank_k(hist)  
emit_ids(prefix)
```

8 fixed FP32 passes, then gather

3.59 ms

radix forward

12.6× lower time

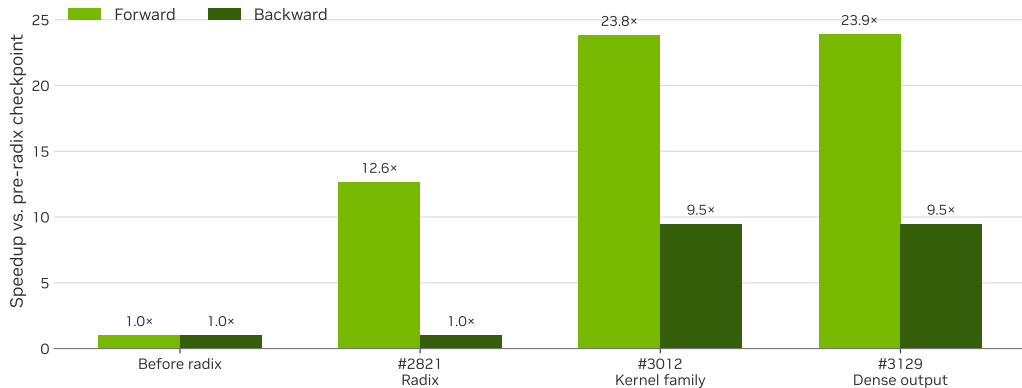
1 warp

per token row

deterministic tie handling

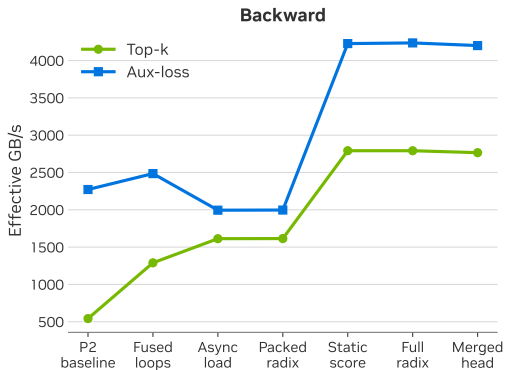
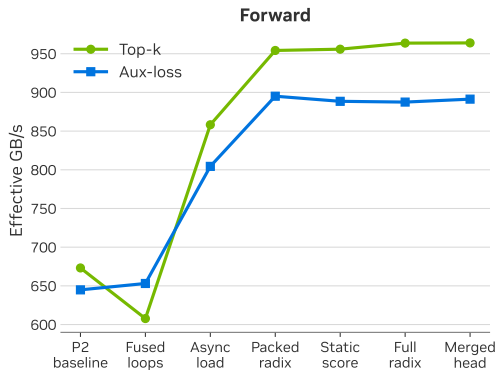
Router Stages

Matched B300 trace-back: 65,536 tokens, 2,304 experts, top- $k = 36$, softmax



P3R Bandwidth

Incremental 8,192-token result for the target 2,304/36 shape



Kernel Design

Coordinated data movement and selection determine router throughput

```
prefetch(tile[0], buffer[0])
for tile in token_tiles:
    prefetch(tile[next], buffer[next])
    wait(buffer[current])
    score(buffer[current])
    radix_topk()
    write_selected()
    swap(current, next)
```

Throughput mechanisms

- Persistent execution maintains SM occupancy across token tiles.
- Double-buffered `cp.async` overlaps score loads with radix selection.
- Packed histograms cut radix counters from 32 to 4; templates remove hot-path branches.
- Two-pass backward removes 36 KiB/block scratch at $E = 2,304$.

Router Bounds

Optimistic limits contextualize the measured 254.5 μ s result

HBM traffic floor

Semantic traffic is approximately 245 MB:

$$245 \text{ MB} / 8 \text{ TB/s} = 30.7 \mu\text{s}.$$

The measured kernel reaches about 12% of raw HBM bandwidth. This counts tensor bytes, not cache-line or instruction overhead.

Compute-issue floor

Eight radix passes touch 151M scores:

$$8 \times 8192 \times 2304 = 151\text{M}.$$

One FP32 op/visit at 75 TFLOP/s/GPU gives an impossible $\sim 2.0 \mu\text{s}$ floor. Real visits include masks, shifts, histograms, shuffles, scans, and dependencies.

Dense Routing Map

Selected IDs replace the dense bool map

Compact representation

Bool $[T, E]$	2,304 B/token
int16 $[T, K]$	72 B/token

$$\frac{E}{2K} = \frac{2304}{72} = 32 \times .$$

Backward also traverses only the K selected IDs; it no longer asks all E experts to test a K -entry membership list.

```
route_ids = int16[T, K]
probs = fused_router(logits,
                     dense_route_out=route_ids)
hybrid_ep.dispatch(
    tokens, dense_route=route_ids, probs=probs
)
```

One compact producer-to-consumer contract connects router, scan, dispatch, and backward without reconstructing the bool map.

HybridEP Optimization



Presentation outline

HybridEP Optimization

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization
- 5 HybridEP Optimization**
- 6 MCore Integration
- 7 End-to-End Results

HybridEP Pipeline

Sparse routing reduces metadata work, local work, and combine latency

Metadata

Route IDs $\rightarrow$ scan/bitsets



Routing

Dispatch $\rightarrow$ permute



Execution and return

Expert MLP $\rightarrow$ combine $\rightarrow$ unpermute

Compact metadata

Avoid bool $[T, E]$ and repeated membership tests.

Sparse local execution

Skip inactive expert slots in permute and unpermute.

Pipeline configuration

Tune pull depth, grouping, and reduction batch jointly.

Probability Transport

Selected probabilities remain required for weighted activations and gradients

```
# keep mathematical state
ids, weights = router(logits)

# communicate only what this owner needs
for route in selected_routes:
    send(token[route], weights[route])

# backward reuses the selected weights
grad_logits = router_backward(ids, weights, grad)
```

103.0 μs

dispatch with
probability payload

controlled B300 NVL8

249.4 μs

combine with
probability payload

pull + reduce

94.4 μs

without prob payload
+9%

210.0 μs

without prob payload
+19%

The “without probability” rows bound transport opportunity; training still preserves selected weights and their backward dependencies.

Ballot Permutation

4.5 active local experts/token leave 86% of slots empty

```
active = route_map[token * E_local + lane] > 0;
mask = __ballot_sync(FULL_MASK, active);

while (mask) {
    expert = __ffs(mask) - 1;
    mask &= mask - 1;
    peer = __shfl_sync(FULL_MASK, my_peer, expert);
    move_selected_token(peer);
}
```

4.27×

permute

396.1 → 92.7 μ s

Threading policy

2.04×

unpermute

269.5 → 131.8 μ s

Use 32 threads/token at latent $H = 512$. Larger hidden sizes retain wider thread groups to expose memory-level parallelism.

Dense Route Scans

Build expert and rank membership once; use bit tests downstream

```
local_mask = 0
rank_mask = 0

for slot in range(K):
    expert = dense_route[token, slot]
    local_mask |= 1 << (expert % E_local)
    rank_mask |= 1 << owner_rank(expert)

if local_mask & (1 << expert):
    process_active_expert()
```

1.58×

Isolated 512-thread scan

525.6 → 331.7 μ s

The scan consumes compact $[T, K]$ route IDs directly; bool route-map reconstruction is unnecessary.

No permute-fusion metadata is included in this isolated comparison.

Combine Configuration

Tuned for 2,304 experts and validated at EP72

```
for group in [1, 2, 4, 8]:  
    for pull_depth, reduce_batch in candidates:  
        time(combine)  
        time(combine + unpermute)  
        verify_correctness()  
  
select shallowest queue that hides pull latency  
validate the selected tuple in the EP72 launch
```

Parameter	NVL8 study	2304E / EP72
Combine SMs	32	32
G2S stages	64	72
S2G stages	8	8
Tokens/group	2	2
Reduce batch	16	72

The EP72 tuple matches the wider
communication domain; neither tuple is
universal.

Combine Pipeline

Group 2 balances parallelism with FIFO lookahead

G2S fetch
Remote TMA reads



Shared FIFO
32 slots/pipeline



Reduction
FP32 accumulation



S2G store
Write result

Lookahead

32 FIFO slots / ~ 8 source
ranks ≈ 4 prefetched tokens.

Saturation

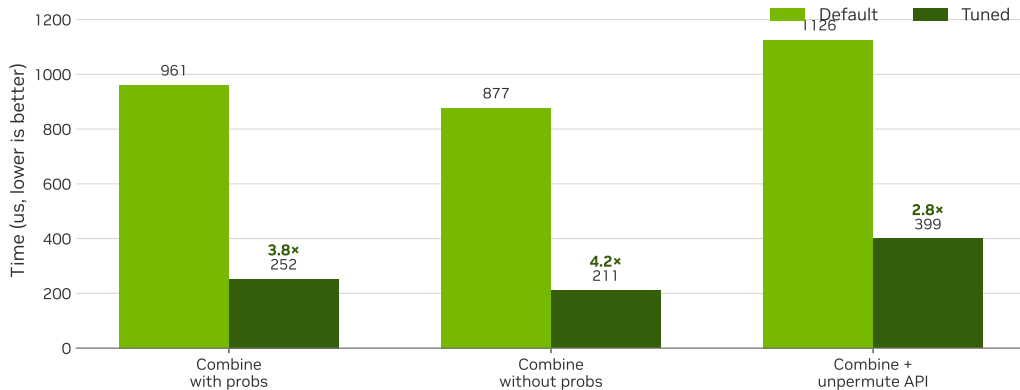
G2S 64 $\rightarrow$ 128 changes 246.1
to 245.0 μ s.

SM allocation

Group 2 matches group 1
throughput with 32 rather
than 64 SMs.

Combine Tuning

Controlled H=512, local-E=32, top-k = 36 comparison



Dispatch vs. Combine

Dispatch is asynchronous; combine synchronizes, reduces, and stores

Dispatch / push

116.8 μ s

650 GB/s

58% long-scoreboard stall

0.05% barrier stall

Combine / pull + reduce

254.4 μ s

271 GB/s

27% long-scoreboard + 26% wait

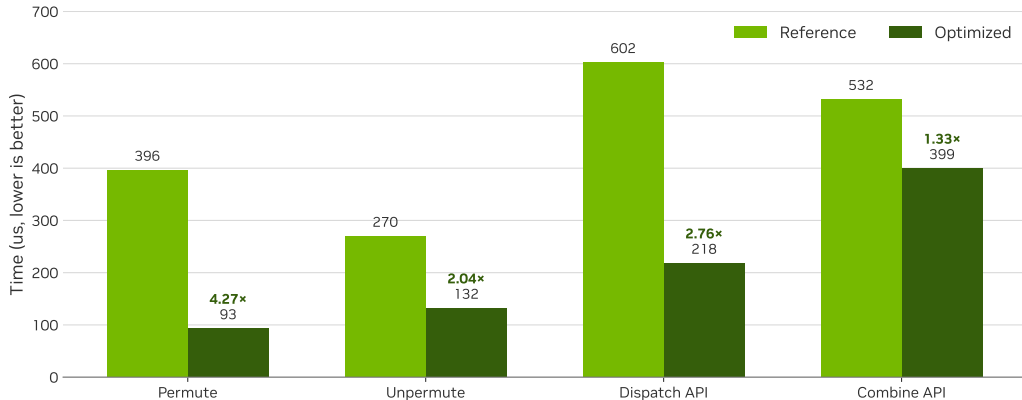
15% barrier stall

Separate combine and unpermute

Fusing them serializes the consumer behind slowly produced chunks: 434 μ s fused versus 381 μ s standalone, about 14% slower.

Microbenchmarks

B300 NVL8, H=512, 8,192 tokens/rank, local-E=32, top-k = 36



Custom All-Gather

Direct registered-buffer writes reduce intermediate copies

~15%

historical isolated
gain

custom all-gather vs
NCCL

32×

logical metadata
reduction

bool $[T, E]$ to int16 $[T, K]$

When it applies

- Writes directly into HybridEP registered buffers.
- Avoids a separate NCCL-buffer-to-PyTorch-tensor copy.
- Benefit is collective- and topology-specific.
- Compact metadata reduces bytes before collective choice matters.

Scope: an isolated communication optimization; the full-model gain is not attributed to all-gather alone.

Design Rejections

Profiling narrows the feasible design space

Direct permute

Repeated remote writes and address work moved the bottleneck into dispatch; the saved 92 μ s permute did not compensate.

Fused combine

Producer-consumer serialization left the best fused path 14% slower than standalone.

Warp-pruned scan

Extra bitset initialization and coordination cost more than the skipped ballots on key shapes.

MCore Integration



Presentation outline

MCore Integration

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization
- 5 HybridEP Optimization
- 6 MCore Integration**
- 7 End-to-End Results

Megatron Core

Integration boundaries

Collaborating workstreams

- Paged stash for bounded activation storage.
- Full-iteration CUDA graph capture and replay.
- 1F1B MoE communication/compute overlap.
- Capture-safe memory release without deferred-free inflation.

Delivered plumbing

```
if te.has_dense_route_output() and \
    dispatcher.accepts_dense_route():
    route = int16[T, K]
    fused_router(..., route_out=route)
    hybrid_ep(..., dense_route=route)
else:
    compatibility_bool_route()
```

Capability detection preserves backward compatibility.

End-to-End Results



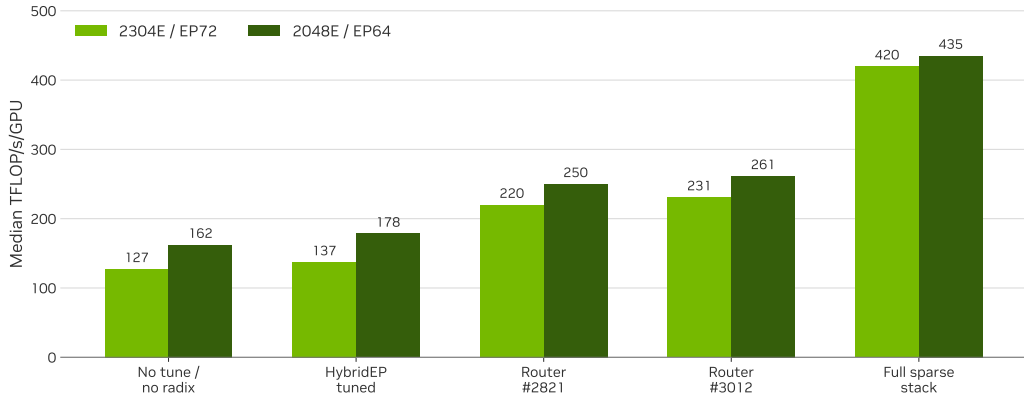
Presentation outline

End-to-End Results

- 1 Introduction
- 2 Target Workload
- 3 Initial Analysis
- 4 Fused-Router Optimization
- 5 HybridEP Optimization
- 6 MCore Integration
- 7 End-to-End Results**

Full-Model Stages

OCI-AGA GB300, 1F1B + paged stash + full-iteration graph; median after warmup 5



Full-Model Throughput

Cross-layer optimization: router, compact routes, sparse preprocessing, bitsets, and tuned combine

419.7

2304E / EP72 median

TFLOP/s/GPU

127.2 no-tune baseline; 3.30×

435.1

2048E / EP64 median

TFLOP/s/GPU

162.2 no-tune baseline; 2.68×

530 / 423

iterations observed

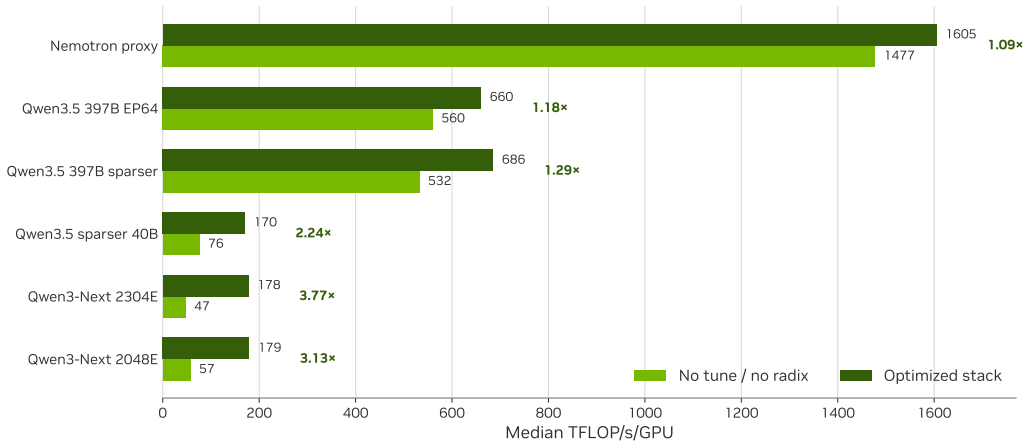
2304E / 2048E full-stack runs

Matched stack

Both shapes use the same 26.06 image family, mock data, MBS 3, 24 layers, full 1F1B, paged stash, full-iteration graph, and the wgrad fix that disables the CuTe DSL fused GroupedMLP wgrad subpath. Isolated kernel speedups are not added together.

MoE-Only Matrix

OCI-AGA GB300; 155 post-warmup iterations; matched full graph + paged stash



MoE-Only Scaling

Generalization across wide, sparse expert shapes

3.77×

Qwen3-Next 2304E

47.2 → 177.9

3.13×

Qwen3-Next 2048E

57.4 → 179.4

2.24×

Qwen3.5 sparser 40B

76.1 → 170.3

Limiting case

Nemotron's proxy shape improved only 1.09×: when expert compute is already large, router and sparse communication are a smaller fraction of the module. This matrix is MoE-only, without 1F1B; full graph and paged stash are matched.

Conclusions

Key findings and applicability limits

Findings

- Radix selection resolves the pathological large- E , large- K case.
- Compact int16 routes reduce logical metadata $32\times$.
- Ballot skips inactive local experts.
- Combine performance requires coordinated queue, group, and batch tuning.
- Gains generalize most strongly to sparse, wide expert shapes.

Scope

- Do not aggregate isolated speedups into end-to-end predictions.
- The EP72 tuple is not a universal default.
- Training retains selected probability transport.
- Separate this work from graph, stash, and 1F1B contributions.
- Distinguish MoE-only and full-model measurements.

Sparse Path

Selected routes remain compact from routing through replay

Select

TE fused router

Represent

int16 $[T, K]$

Exchange

HybridEP

Pipeline

Combine

Replay

Megatron Core



Radix + persistent

Avoid $[T, E]$ expansion

Slice probabilities;
bitsets

Tune stages /
groups / batch

Capture-safe full
stack

23.9×

router forward

pre-radix → dense output

1.84×

HybridEP API total

1,133.7 → 617.1 μ s

419.7

full-model TFLOP/s/GPU

2,304 experts / EP72

Appendix



Presentation outline

Appendix

8

Appendix

Combine Sensitivity

EP72: group 2 retains throughput with half the SMs

Configuration	Time	Output BW	Interpretation
Group 1 / 64 SMs	252 μ s	265 GB/s	Full FIFO depth; only half of warp resources are utilized
Group 2 / 32 SMs	254 μ s	263 GB/s	Same throughput with half as many SMs
Group 4 / 32 SMs	502 μ s	133 GB/s	Insufficient FIFO depth to hide pull latency
G2S 64 $\rightarrow$ 128	246 $\rightarrow$ 245 μ s	flat	Additional queue depth provides no measurable gain

Router Timings

65,536 tokens, 2,304 experts, top- $k = 36$, softmax

Checkpoint	Forward	Backward	Output
Before large top- k support	45.358 ms	3.552 ms	sparse bool
PR #2821	3.587 ms	3.553 ms	sparse bool
PR #3012	1.904 ms	0.375 ms	sparse bool
PR #3129	1.899 ms	0.375 ms	dense int16

Interpretation

PR #2821 changes the selection algorithm; PR #3012 redesigns forward and backward; PR #3129 changes the API representation without regressing the target forward path.

Sources

The presentation is reproducible from local measurements and official model sources

Topic	Primary source
Persistent content memory	<code>../sparser_moe_final_presentation_content.md</code>
Router trace-back	<code>data/router_trace_back.csv</code> ; <code>data/router_fix_p3R_*.csv</code>
HybridEP	<code>hybrid-ep-sparse-opt-new.md</code> ; <code>hybrid_ep_rebased_bw_results.md</code>
Dense scan	<code>isolated-scan-bench.md</code>
Full-model sweep	<code>qwen3_next_2606_image_sweep_20260728/README.md</code>
MoE-only matrix	<code>moe_perf_canonical_matrix_20260730/README.md</code>
Open models	Mixtral; DeepSeek-V3; Nemotron 3 Ultra; DeepSeek-V4-Pro; Kimi K2; Kimi K3
Hardware roofs	NVIDIA HGX B300; NVIDIA HGX AI Factory components

Discussion

